

FLASH: A “Generate Once, Synthesize Many” Framework for Synthetic Anomaly Generation in Industrial Anomaly Detection

Anonymous WACV Applications Track submission

Paper ID 2035

Abstract

Synthetic anomaly generation helps expand industrial anomaly datasets when real defects are scarce or unavailable. Existing approaches lie at two extremes: procedural approaches are fast but struggle to represent complex anomalies, while generative approaches produce diverse defects but require costly per-sample generation. We present FLASH, a framework that decouples defect generation from anomaly synthesis under a “generate once, synthesize many” paradigm. Given only normal images, FLASH uses Vision Language Model (VLM) guidance and an image-generation model to produce a small set of defect images, from which it extracts, validates, and banks reusable defect patches. For synthesis of anomalous images, Object Boundary Suppression (OBS) first identifies the probable foreground object-aware region of the host image, while Multi-Resolution Spectral Pyramid (MRSP) noise generates diverse, size-controllable masks that determine the defect location and spatial extent. It then composes a large and diverse synthetic anomalous image set by localizing the defect region, sampling size-controllable placement masks and seamlessly blending retrieved defects onto new defect-free images without further need for image generation. Experiments on the MVTec AD 2 dataset show that FLASH-generated anomalies nearly close the calibration gap on real defects, reaching 78.1% image-level F1 against an 83.6% real-anomaly upper bound and providing the most consistent calibration transfer across detectors among procedural and generative alternatives. Moreover, FLASH synthesizes anomalies more than $11.95\times$ faster than per-sample generative approaches.

1. Introduction

In manufacturing, production lines predominantly yield defect-free products, while anomalous samples are rare, diverse, and expensive to collect and annotate [5]. This makes synthetic anomaly generation an attractive alternative for

constructing anomalous data from normal images [50, 52]. Existing approaches occupy two extremes, computationally faster procedural and feature-based methods, and the slower but contextually coherent generative methods. The first set of methods is limited in their ability to represent complex and semantically meaningful defects [29, 30] whereas the second, though closer to the target domain, require substantially more time and computation per generated sample [22, 26]. We therefore aim to develop a framework that lies between these two extremes: retaining the fast generation and computational efficiency of procedural synthesis while obtaining diverse synthetic anomalies through limited generative inference. We introduce FLASH, a framework designed around this trade-off. It generates a small set of defect instances once and reuses them to synthesize a substantially larger anomaly dataset, thereby reducing generation time while operating without any real defect references. The resulting synthetic anomalies are context-specific, thus serving as a viable surrogate in the absence of real defects for downstream anomaly detection, including decision-threshold calibration [54].

FLASH achieves this through a five-stage approach. Its factorized design allows a limited set of generated defects to be reused across normal images, balancing generation speed, computational efficiency, and synthetic anomaly utility. We evaluate FLASH on MVTec AD 2 [20], a challenging benchmark with diverse conditions and anomaly types beyond conventional structural defects, including subtle foreign contamination. Such anomalies challenge procedural synthesis, where predefined noise or feature perturbations have limited capacity to model the appearance, context, and spatial characteristics of externally introduced materials, making MVTec AD 2 a stringent test of whether reference-free synthesis can move beyond simple procedural defects.

2. Related Work

Synthetic anomaly generation encompasses procedural methods that prioritize efficient synthesis and generative ap-

proaches that emphasize contextual fidelity, forming a persistent procedural–generative trade-off.

2.1. Procedural Anomaly Generation

Procedural methods establish a simple route to reference-free anomaly synthesis by transforming normal images through patch relocation [29], blending [49], and Poisson cloning [43]. Noise-based formulations extended this principle to scalable defect formation using Perlin masks and sampled textures [35, 52], with subsequent work improving object constraints [51] and perturbation subtlety [8]. Their low computational cost makes them attractive for large-scale synthesis; however, the generated defect space remains tied to predefined perturbations or borrowed textures, limiting their ability to represent complex context-dependent anomalies such as those arising from real manufacturing process error.

2.2. Generative Anomaly Generation

Generative models address the above limitations by broadening the diversity of defects that can be represented. GAN-based approaches introduced defective counterparts [32], controllable synthesis [53], and defect transfer from limited anomalous samples [13]. Diffusion-based approaches [39] further enabled anomaly embeddings [22], joint image-mask generation [26], controllable anomaly strength [55], inpainting [48], cross-category transfer [45], and spatial conditioning [18]. Vision-language models subsequently added contextual reasoning to anomaly analysis [17, 23, 56] and synthetic defect generation [21, 25, 27]. These advances broaden the range of defects that can be generated, but generative inference typically must be repeated for each synthesized sample, making the process slow and computationally expensive, and several approaches also rely on real anomalous references [9] – placing generative methods at the opposite end of the synthesis spectrum from procedural approaches in terms of diversity versus efficiency.

2.3. Industrial AD Benchmarks and Detectors

The practical value of synthetic anomalies depends on whether they provide useful anomalous evidence for downstream industrial anomaly detection. Most synthetic anomaly generation methods are benchmarked on MVTec AD [5] and VisA [57]. However, these datasets are well-explored, primarily feature simple procedural defects, and often fail to expose the limitations of synthetic anomalies. In contrast, MVTec AD 2 [20] is a far more demanding industrial dataset featuring transparent objects, challenging illumination, and foreign contamination. Despite its realism, it remains underutilized, making it an ideal testbed for evaluating synthetic anomalies beyond basic procedural defects.

Downstream anomaly detection methods span several architectural paradigms, including embedding-based (PaDiM

[11], PatchCore [42]), feature-mapping (SimpleNet [30]), student-teacher networks (EfficientAD [4]), reconstruction-based frameworks (Reverse Distillation [12]), and modern foundation-model approaches (Dinomaly [19], AnomalyDINO [10], SuperADD [40]). However, existing benchmarks predominantly evaluate models using decision thresholds calibrated on real anomalies, an assumption that breaks down in practical zero-shot or unsupervised industrial settings where real anomalies are unavailable prior to deployment.

3. Methodology

Industrial anomaly detection focuses on normal data as defective samples are rare, unevenly distributed across defect types, and expensive to collect and annotate at pixel level [5]. FLASH addresses this in its strictest form: given only normal images from a category, it generates anomalous images with pixel-accurate segmentation labels without using any real defect image, defect mask, or human annotation. Throughout the pipeline, I_n denotes the Normal Image, I_a the Generated Anomaly Image, M_d the Defect Mask, C_d the Defect Crop, B the Category-wise Semantic Defect Bank, I_h the Different Normal Host Image, Ω the Object-Aware region, M_f the MRSP noise-based placement mask, I_{syn} the Final Synthetic Anomaly Image, and M_{gt} its corresponding ground-truth mask.

Existing pipelines are predominantly mask-first: a defect mask is defined before the defect is generated, after which the model is required to conform its output to the prescribed region [51]. Diffusion-based approaches follow this formulation by conditioning image generation on predefined defect regions [48]. This creates a mask-drift problem: the generated defect may not fully occupy the prescribed mask or may extend beyond it, causing disagreement between the anomaly and its label. Unlike other approaches [18], FLASH generates the anomalous frame first and then recovers the actual Defect Mask M_d from the generated image I_a . This mask can then be transferred onto a different Normal host Image I_h and placed within the MRSP mask M_f , where its location and spatial extent can be varied without distorting the recovered defect.

Figure 1 presents the five-stage FLASH architecture. Stage 1 uses a VLM reasoning about the entire image and a Config File to guide the image-generation model in producing I_a from I_n . Stage 2 recovers M_d using DiffMask and extracts the Defect Crop C_d , while Stage 3 validates the crop and stores it in the Category-wise Semantic Defect Bank B . Stage 4 processes the Different Normal Host Image I_h using OBS and MRSP to obtain the Object-Aware region Ω and placement mask M_f . Stage 5 retrieves a defect from B and adaptively blends it within M_f to produce I_{syn} and M_{gt} from I_h . Thus, I_n serves as the donor for defect creation, while I_h serves as the host for subsequent synthe-

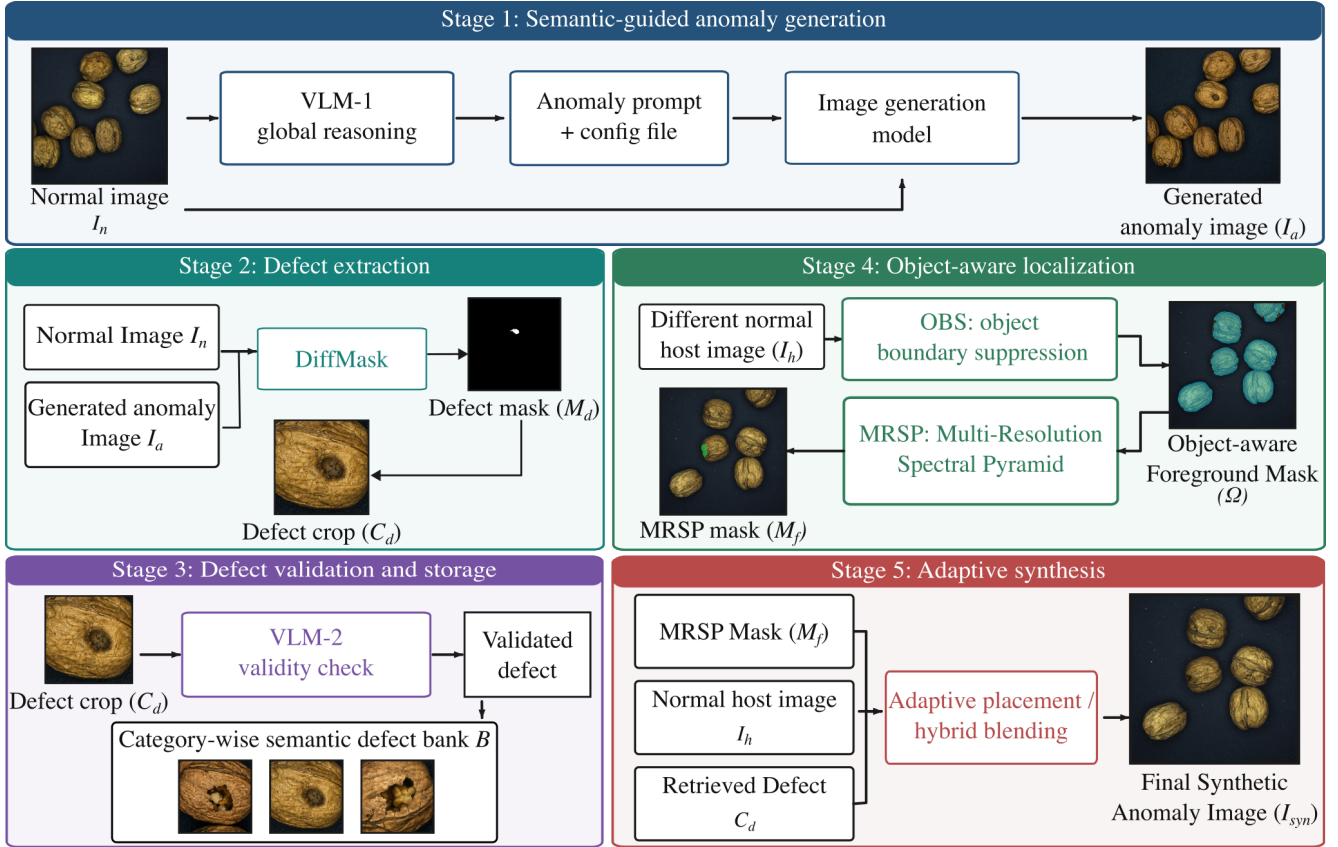


Figure 1. Five-stage FLASH architecture. Stages 1–3 generate, extract, validate, and store reusable defects, while Stages 4–5 localize and synthesize retrieved defects onto separate normal host images.

sis; the detailed operations of each stage are described in the following sections.

Stages 1–3 are executed once per category to construct the defect bank, while Stages 4–5 execute for each output image. Thus, the image-generation model is not invoked for every synthetic sample, unlike fully generative pipelines whose generation cost scales with corpus size [48]. The per-image generation instead relies on lightweight placement, harmonization and blending, allowing FLASH to retain the efficiency of procedural synthesis while using limited generative inference to obtain diverse contextual defects. The resulting defect bank can then be reused across normal host images, with MRSP mask providing intra-image variation for additional diversity. An overview of the complete FLASH pipeline across all eight MVTeC AD 2 categories, showing host images, OBS regions, placement masks, extracted defects, ground-truth masks, and final synthetic anomalies, is shown in Figure 3.

3.1. Semantic-Guided Anomaly Generation

Starting from a Normal Image I_n , the first challenge is to specify what anomaly should be introduced before pass-

ing the image to the image-generation¹ model. Rather than manually defining this prompt, VLM-1² performs global reasoning over I_n to identify a plausible defect type, its material context, and then generates an Anomaly Prompt. This prompt is augmented by a Config File that provides dataset context, category-wise defect types, and hard generation constraints. Crucially, it enforces photometric preservation with respect to I_n , ensuring that lighting elements including shadows, reflections, and colour distribution remain unaltered and strictly prohibits changes to exposure, contrast, or white balance. As shown in Figure 1, the Anomaly Prompt, Config File, and I_n are provided to the image-generation model to generate the Anomaly Image I_a . Thus, VLM-1 provides the semantic specification of the anomaly, while the Config File provides dataset-specific context and generation constraints. Since the image-generation model re-generates the entire image with anomaly, it may exhibit scale, spatial-offset, and pixel-level correspondence differences from I_n , motivating the Defect Extraction stage.

¹ChatGPT [33]

²Qwen2.5-VL-7B-Instruct [3]

3.2. Defect Extraction

DiffMask frames the difference between the Normal Image I_n and the generated Anomalous Image I_a as a signal-recovery problem in which alignment, illumination, and reconstruction noise act as nuance variation, while the introduced defect constitutes the signal. The two images are first registered using their edge maps to establish structural correspondence, followed by photometric adjustment.

As detailed in Algorithm 1, the registration is refined through similarity alignment, ECC refinement [14], and when required, feature-based [31] and dense-flow correction [15] to absorb residual geometric drift. The registered images are then compared using a tolerance band to suppress small registration errors. Five complementary cues measure luminance, chromaticity, gradient structure, low-frequency appearance, and fine-detail changes, which are fused into the difference score S . The complete registration and difference-based extraction process is illustrated in Figure 2.

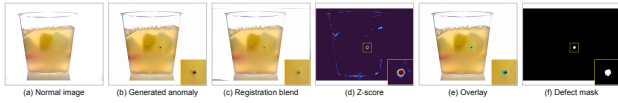


Figure 2. DiffMask-based defect extraction from registered normal and generated anomaly images.

Rather than applying a fixed global threshold to S , DiffMask evaluates each pixel relative to its local reconstruction noise. For a pixel p , let $\mathcal{N}(p)$ denote its local neighbourhood. The local deviation is computed as

$$Z(p) = \frac{S(p) - \mu_{\mathcal{N}(p)}}{\max(\sigma_{\mathcal{N}(p)}, \epsilon)}, \quad (1)$$

where $Z(p)$ is the local z-score, $S(p)$ is the fused difference score, $\mu_{\mathcal{N}(p)}$ and $\sigma_{\mathcal{N}(p)}$ are the mean and standard deviation of S within $\mathcal{N}(p)$, and ϵ prevents unstable normalization when the local variance is near zero. This makes the defect decision dependent on how strongly a pixel deviates from its surrounding reconstruction noise rather than on its absolute difference magnitude.

The resulting Z map is separated into strong and weak candidates using absolute and adaptive thresholds. Hysteresis thresholding [7] retains weak responses only when connected to strong defect evidence, suppressing isolated noise while preserving faint defect extensions. The surviving regions are then refined, ranked by accumulated evidence, and subjected to direction-matched completion to recover weak portions that exhibit the same characteristic change as the detected defect. The resulting Defect Mask M_d is used to extract the Defect Crop C_d from I_a with surrounding substrate context for subsequent validation and storage.

Algorithm 1 DiffMask

Require: Normal Image I_n , Generated Anomaly Image I_a

Ensure: Defect Mask M_d , Defect Crop C_d

- 1: Convert I_n and I_a to structural representations and obtain an initial similarity transform using scale-translation search.
- 2: Refine registration using coarse-to-fine ECC alignment; if the alignment score is insufficient, evaluate feature-based similarity registration and retain the better solution.
- 3: Apply smoothed dense-flow refinement to compensate for residual local misalignment.
- 4: Photometrically match the registered I_n to I_a using local gain-offset fitting and construct a tolerance band around the reference response.
- 5: Compute luminance, chromaticity, gradient/edge structure, low-frequency appearance, and sharpness/fine-detail residuals, and fuse them into the difference score S .
- 6: Estimate robust global and local neighborhood statistics of S and compute the local z-score $Z(p)$.
- 7: Generate strong and weak candidates using absolute and adaptive thresholds, and apply hysteresis thresholding to retain weak candidates connected to strong defect evidence.
- 8: Remove isolated responses and refine surviving regions; rank connected components by accumulated evidence and retain valid ones.
- 9: Recover weak defect continuations using direction-matched completion and obtain the final defect mask M_d .
- 10: Extract the defect crop C_d from I_a with surrounding substrate context.
- 11: **return** M_d, C_d

3.3. Defect Validation and Banking

DiffMask recovers candidate defect regions, but its difference cues can also flag random texture variations or reconstruction artifacts that do not correspond to valid defects. Therefore, the extracted Defect Crop C_d is passed to VLM-2³, for semantic validation. VLM-2 evaluates C_d together with its surrounding substrate and accepts it only when the detected region represents a plausible category-specific defect; otherwise, the crop is rejected. A contrast gate additionally verifies that the candidate defect has sufficient contrast relative to its surrounding substrate. Each accepted crop becomes a *Validated Defect* and is stored in the Category-wise Semantic Defect Bank B , indexed by category and defect type. Thus, entries are not selected solely based on pixel-level differences; they must also pass VLM-

³Qwen2.5-VL-7B-Instruct [3]

2's semantic validity check and correspond to a valid defect.

Once a Defect Crop C_d is accepted into B , its generative cost is incurred only once. The stored defect can then be retrieved and reused across different Normal Host Images I_h , locations, and spatial extents, allowing a limited set of generated and validated defects to produce many distinct synthetic anomaly instances without invoking the image-generation model again.

3.4. Object-Aware Localization

After constructing the Category-wise Semantic Defect Bank B , FLASH must determine where a retrieved defect can be placed on a different Normal Host Image I_h . The localization stage consists of two components. First, Object Boundary Suppression (OBS) estimates the physically meaningful foreground region Ω of I_h . Second, Multi-Resolution Spectral Pyramid (MRSP) generates a coherent placement field within this region. The resulting MRSP mask M_f controls the location and spatial extent of the retrieved defect without altering its appearance. This allows the same defect to be reused across different host images, locations, and spatial extents.

3.4.1. Object Boundary Suppression (OBS)

The procedural noise field has no knowledge of the physical object in I_h and can therefore place a defect on the background. The key intuition behind OBS is to combine appearance and texture cues: pixels that differ from the background (estimated from the image boundary) are likely to belong to the foreground object [1], while local texture variation can recover object regions that have weak colour separation.

As summarized in Algorithm 2, for every Host Image, it first estimates the background from the median colour of the image-border pixels in CIELAB space. A colour-distance response is then Otsu-thresholded [34], while a local standard-deviation response provides an independent texture cue. The two responses are combined using a logical OR and subsequently refined using morphological operations [44] and connected-component area filtering. A degeneracy guard handles cases in which the resulting foreground estimate is either empty or covers nearly the complete image.

The resulting Object-Aware region is defined in Eq. 2

$$\Omega = \mathcal{R} \left(M_{\text{col}}^{\text{Otsu}} \vee M_{\text{tex}}^{\text{Otsu}} \right), \quad (2)$$

where $M_{\text{col}}^{\text{Otsu}}$ and $M_{\text{tex}}^{\text{Otsu}}$ are the colour- and texture-based foreground responses, respectively, and $\mathcal{R}(\cdot)$ denotes morphological refinement and connected-component filtering. The OR operation allows a region to be retained when either colour or local texture provides sufficient evidence of the object.

Algorithm 2 Object Boundary Suppression

Require: Different Normal Host Image I_h

Ensure: Object-Aware region Ω

- 1: Convert I_h to CIELAB space and estimate the background colour from the median of the image-border pixels.
 - 2: Compute the colour-distance response from the estimated background and local grayscale standard-deviation map.
 - 3: Apply Otsu thresholding to obtain the colour response M_{col} from the estimated background and the texture response M_{tex} from the grayscale standard-deviation map.
 - 4: Combine the responses using a logical OR.
 - 5: Refine with morphological opening, closing and hole filling.
 - 6: Remove connected components below the minimum area threshold; retain the largest component if area filtering removes all candidates.
 - 7: Apply the degeneracy guard for empty or near-full-image estimates to obtain Ω (Eq. 2).
 - 8: **return** Ω
-

3.4.2. Multi-Resolution Spectral Pyramid (MRSP)

Given the Object-Aware region Ω , FLASH must determine a compact region within the object where a retrieved defect can be synthesized. This region should be spatially coherent while allowing both its location and size to vary across generated samples. MRSP constructs such a placement field by combining two noise components generated across multiple spatial resolutions [6]: multi-resolution spectral noise [16] and bilinear upsampling noise [28]. Low-frequency components establish the coarse blob structure, whereas higher-frequency components introduce irregularity along its boundaries.

As summarized in Algorithm 3, MRSP generates spectral noise at multiple resolutions using $1/f^\alpha$ amplitude weighting [16] and uniformly sampled random phase. Here, f denotes spatial frequency and α controls the frequency-dependent amplitude decay. Each spectral response is transformed to the spatial domain using an inverse Fourier transform; lower-resolution responses are then bilinearly upsampled to the host resolution and normalized. The resulting multi-resolution responses are combined using persistence weighting [35] and energy normalization to obtain the placement field F_{MRSP} . Thus, spectral synthesis controls the frequency structure, while bilinear upsampling brings each resolution to a common spatial scale.

The target extent or size is derived from normal-image statistics. Object coverage, the number of visible object instances, and the dataset-specific defect-coverage prior determine the target coverage c . Hence, the stochastic MRSP

Algorithm 3 Multi Resolution Spectral Pyramid Noise

Require: Host dimensions (H, W) , Object-Aware region Ω , target coverage c

Ensure: MRSP placement mask M_f

- 1: Construct the multi-resolution frequency grids and generate spectral amplitudes using $1/f^\alpha$ weighting.
- 2: Sample a uniformly random phase for each spectral level and form spectral response using the inverse Fourier transform.
- 3: Bilinearly resize each response to (H, W) and normalize it.
- 4: Combine the normalized responses using persistence weighting and energy-normalize the combined response to obtain F_{MRSP} .
- 5: Compute the object-conditioned quantile threshold Q_{1-c} from $F_{\text{MRSP}}[\Omega > 0]$.
- 6: Threshold F_{MRSP} , gate by Ω , and retain the largest connected component as M_f (Eq. 3).
- 7: **return** M_f

realization controls *where* the placement appears, while c controls *how much* of the object it occupies.

A threshold computed over the complete image can select strong responses outside Ω , while direct intersection with Ω can fragment or remove the intended placement region. FLASH therefore computes the threshold only from responses within Ω . Let $Q_q(\cdot)$ denote the q -quantile operator. The final placement mask is defined in Eq. 3

$$M_f = L[(F_{\text{MRSP}} > Q_{1-c}(F_{\text{MRSP}}[\Omega > 0])) \wedge \Omega], \quad (3)$$

where F_{MRSP} is the normalized MRSP placement field, Ω is the Object-Aware region obtained from OBS, $Q_{1-c}(\cdot)$ is the $(1 - c)$ -quantile operator, c is the target object-region coverage, and $L(\cdot)$ retains the largest connected component. Computing the quantile within Ω makes the specified coverage relative to the object rather than the complete image.

The resulting M_f provides a single coherent placement region for Stage 5. Different MRSP realizations vary the defect location, while different target coverages vary its occupied object area, providing controllable spatial and size diversity without additional generative inference.

3.5. Adaptive Synthesis

This final stage combines the Normal Host Image I_h , MRSP placement mask M_f and retrieved defect crop C_d to produce the final synthetic anomaly I_{syn} . The objective is to reuse the generated defect while varying its placement and size on a new normal host. Thus, C_d determines *what* is synthesized, while M_f determines *where* it is synthesized.

As summarized in Algorithm 4, FLASH uses an original placement mode and an adaptive placement mode. The

Algorithm 4 Adaptive Placement and Hybrid Blending

Require: Host Image I_h , Defect Crop C_d , MRSP Mask M_f , Object Region Ω

Ensure: Synthetic Anomaly I_{syn} , Ground-Truth Mask M_{gt}

- 1: Extract the MRSP blob centroid (x_f, y_f) , area A_f , and orientation ϕ_f .
- 2: Initialize the defect using its source-relative scale and sample rotation and horizontal flip.
- 3: **if** adaptive placement **then**
- 4: Estimate defect orientation ϕ_d and align with ϕ_f ; isotropically scale the defect according to the MRSP blob coverage.
- 5: Anchor the defect at (x_f, y_f) or the deepest interior point of M_f ; reduce its scale until the defect mask satisfies the MRSP containment constraint.
- 6: **else**
- 7: Place the defect at (x_f, y_f) using its source-relative scale.
- 8: **end if**
- 9: Place the transformed defect on I_h and constrain its mask by $M_f \cap \Omega$; harmonize the localized crop with the host substrate in Lab space.
- 10: Set M_{gt} to the placed defect mask and compute its equivalent radius r_{eq} .
- 11: **if** $r_{\text{eq}} < \tau$ **then**
- 12: Apply feathered alpha blending.
- 13: **else**
- 14: Extract the defect core and construct a substrate collar around M_{gt} to form M_c ; apply Poisson blending within M_c , falling back to alpha blending if it fails.
- 15: **end if**
- 16: **return** $I_{\text{syn}}, M_{\text{gt}}$

original mode places the defect at the MRSP blob centroid while preserving its source-relative scale. The adaptive mode additionally uses the blob area, principal orientation, centroid, coverage, and containment to adapt the defect to the selected MRSP region. The defect is isotropically scaled according to the target coverage, aligned with the MRSP orientation, anchored at the blob centroid or deepest interior point [41], and iteratively reduced when necessary to satisfy the containment constraint. Rotation and horizontal flipping provide additional placement variation while preserving the recovered defect morphology.

The placed crop is then harmonized with I_h in CIELAB space using source-substrate and local host-substrate statistics [38]. FLASH subsequently applies hybrid blending: small adaptive placements use feathered alpha blending [36], while larger placements use Poisson blending [37] with a substrate collar. The collar moves the blending boundary into the surrounding substrate [24], preventing the

blending operation from crossing the defect boundary and reducing visible seams that could otherwise be detected as artificial edges. If Poisson blending fails, alpha blending is used as a fallback. The hybrid operation is defined in Eq. 4.

$$I_{\text{syn}} = \begin{cases} \mathcal{A}(I_h, C'_d), & r_{\text{eq}} < \tau, \\ \mathcal{P}(I_h, C'_d, M_c), & r_{\text{eq}} \geq \tau, \end{cases} \quad (4)$$

where I_{syn} is the final synthetic anomaly, I_h is the normal host image, C'_d is the CIELAB-harmonized defect crop, r_{eq} is the equivalent radius of the placed defect, τ is the small-defect threshold, $\mathcal{A}$ denotes feathered alpha blending, $\mathcal{P}$ denotes Poisson blending, and M_c is the substrate-collar compositing mask.

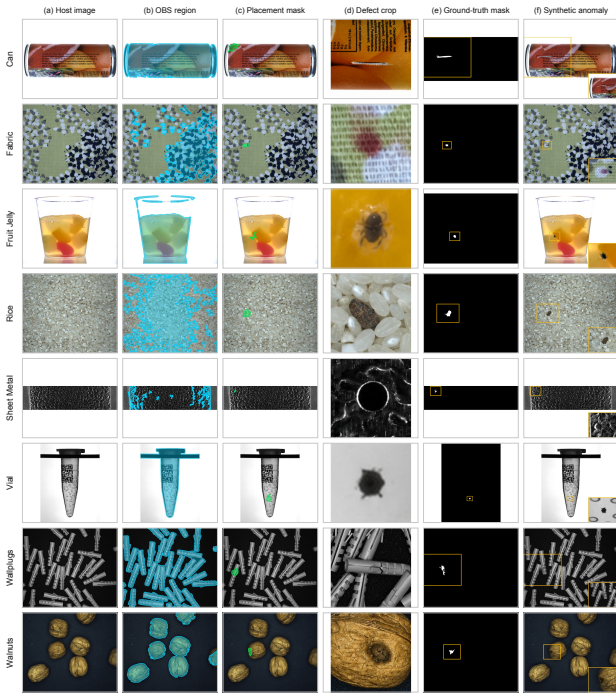


Figure 3. Qualitative results across all eight MVtec AD 2 categories, illustrating the end-to-end FLASH pipeline.

4. Experiments

This section evaluates synthetic anomalies from FLASH as a substitute for real defects in decision-threshold calibration across anomaly detectors and MVtec AD 2 categories, with real-defect calibration as the oracle.

4.1. Experimental Setup

We evaluate FLASH on MVtec AD 2 [20], which contains high-resolution industrial inspection scenarios with varying illumination, overlapping objects, transparent and reflective surfaces, and localized defects. The complete pipeline is executed independently for each category following the *Gen-*

erate Once, Synthesize Many formulation. SuperADD [40] with DINOv3 ViT-H/16+ [46] features is used as the primary downstream detector. To determine whether the calibration behavior transfers beyond a single detector, we additionally evaluate PaDiM [11], PatchCore [42], AnomalyDINO [10] and Dinomaly [19], covering parametric, coreset-based, training-free, and distillation-based architectures, respectively, using Anomalib [2].

For each evaluation, the detector and its memory bank are fitted once and held fixed; only the data used for decision-threshold calibration is varied. The *Real* setting fits the threshold using real test anomalies and therefore serves as an oracle upper bound rather than a competing method. Each synthetic setting instead uses held-out normal samples and its own generated anomalies for calibration and determining the threshold, followed by evaluation on the same real test set. A synthetic source is therefore effective to the extent that its calibrated performance approaches the *Real* reference. All experiments are repeated with three random seeds and the reported results are averaged across seeds.

4.2. Cross-Model Comparison

We compare FLASH with Perlin noise [35, 52] and AnoStyler [47] under the same synthetic-anomaly budget and calibration protocol, with the downstream detector fixed. Perlin represents the procedural, reference-free baseline, blending textures from the Describable Textures Dataset through Perlin-noise masks as implemented in DRAEM [52]. AnoStyler represents the text-driven generative alternative, using a style-transfer network under CLIP-guided textual losses to synthesize defects within procedurally localized foreground masks. This controlled comparison isolates the effect of the synthetic calibration source on transfer to real defects.

Table 1 shows that FLASH provides the most consistent synthetic calibration across the five detectors. At the image level, FLASH retains 91–98% of the corresponding *Real* oracle performance and achieves the best synthetic result for four of the five detectors. At the pixel level, FLASH retains 42–75% of oracle performance and achieves the best synthetic result for four out of the five detectors. For SuperADD, FLASH reaches 78.1% image-level F1 against the 83.6% *Real* oracle, while substantially improving pixel-level F1 over Perlin. Although Perlin obtains a marginally higher image-level F1 for SuperADD, its weaker pixel-level result shows that global anomaly separation does not necessarily yield spatially useful calibration. AnoStyler remains stronger for PaDiM and AnomalyDINO at the pixel level, but exhibits larger variation across detectors. Overall, FLASH provides the most consistent cross-model transfer rather than dominating every individual metric.

Model	Real	Perlin	AnoStyler	FLASH (Ours)
PaDiM	80.37	69.41	62.37	79.11
PatchCore	82.46	67.50	43.02	74.64
AnomalyDINO	81.47	62.19	49.85	77.05
Dinomaly	81.81	60.42	55.37	77.51
SuperADD	83.64	79.12	64.36	78.13

(a) Image F1

Model	Real	Perlin	AnoStyler	FLASH (Ours)
PaDiM	7.63	3.05	5.37	3.89
PatchCore	26.09	14.90	16.08	18.14
AnomalyDINO	33.55	17.47	26.11	13.99
Dinomaly	31.9	15.45	3.35	22.53
SuperADD	51.53	19.44	36.99	38.38

(b) Pixel F1

Table 1. (a) Image-level F1 and (b) pixel-level F1 scores (%) averaged across categories of the MVTec AD 2.

Category	Real	Perlin	AnoStyler	FLASH (Ours)
Can	0.02	0.01	0.00	0.00
Fabric	78.36	18.57	44.30	69.06
Fruit Jelly	56.07	40.02	55.93	55.57
Rice	58.75	6.21	25.36	51.90
Sheet Metal	35.87	2.53	7.79	7.39
Vial	57.05	56.46	46.63	39.41
Wallplugs	54.30	1.98	50.79	17.41
Walnuts	71.86	29.78	65.12	66.28
Mean	51.53	19.44	36.99	38.38

Table 2. Per-category pixel-level F1 scores (%) using SuperADD.

Method	I_a Gen. (s/image)	I_{syn} Gen. (ms/image)	Total (s/category)
Perlin	–	14.4	1.3
AnoStyler	–	14,982.1	1,348.1
FLASH (Ours)	15	586.2	112.8

Table 3. Estimated generation time for I_a and I_{syn} . *Total (s/category)* indicates the total time taken to generate the synthetic images for a category of MVTec AD 2.

11.95 $\times$ speedup or a 91.6% reduction in generation time.

499

5. Conclusion and Future Work

500

The results establish FLASH as an effective approach to bridging the procedural–generative trade-off in synthetic anomaly generation. Across MVTec AD 2, FLASH provides strong and consistent calibration across diverse anomaly-detection architectures and categories, reaching 78.1% image-level F1 for SuperADD against an 83.6% Real oracle upper bound. Its reusable defect banks preserve the contextual diversity of generative anomalies while enabling their transfer across normal hosts, making synthetic anomalies a viable surrogate for real defects in decision-threshold calibration. FLASH also achieves an estimated 11.95 $\times$ speedup, corresponding to a 91.6% reduction in generation time relative to per-sample generative synthesis. Together, these findings establish that FLASH can provide scalable synthetic calibration while substantially reducing dependence on real defect samples.

Future work will focus on reducing the remaining category-dependent calibration gaps, particularly for geometry-dominated and challenging illumination regimes, through increased defect diversity and adaptive defect-bank construction. We will further evaluate FLASH across additional industrial datasets and study how calibration quality evolves with larger synthetic anomaly sets.

524

References

525

- [1] R. Achanta, S. Hemami, F. Estrada, and S. Süsstrunk. Frequency-tuned salient region detection. In *CVPR*, pages 1597–1604, 2009. 5
- [2] S. Akcay, D. Ameln, A. Vaidya, B. Lakshmanan, N. Ahuja,

529

4.3. Category-wise Behaviour and Efficiency

Table 2 examines pixel-level F1 for SuperADD across the eight MVTec AD 2 categories, with *Real* providing the oracle reference. FLASH achieves the highest mean among the synthetic sources and approaches the oracle most closely on several textured-material categories, including fabric, walnuts, rice, and fruit jelly. The improvement is particularly pronounced for rice, where Perlin provides substantially weaker calibration. The remaining categories reveal complementary behavior: AnoStyler performs better on wallplugs, where the anomalies are primarily geometric, while Perlin performs better on vial, a transparent back-lit setting. Sheet metal remains challenging for all synthetic sources under its dark-field illumination and specular structure. The *can* category is a hard case, with the *Real* oracle itself achieving close to 0, and therefore does not provide a meaningful distinction between synthesis strategies. Failure on this category is reported by SuperADD in [40] as well. Overall, the category-wise results support FLASH as a transferable calibration source across diverse defect types while transparently exposing the regimes where alternative synthesis strategies remain advantageous.

Table 3 shows that Perlin and AnoStyler require no separate I_a generation, whereas FLASH incurs this one-time step for defect extraction. Despite this overhead, FLASH has an estimated generation time of 112.8 s per category, compared with 1348.1 s for per-category generative synthesis per category. This corresponds to an approximately

- and U. Genc. Anomalib: A deep learning library for anomaly detection. In *ICIP*, pages 1706–1710, 2022. 7
- [3] S. Bai, K. Chen, X. Liu, et al. Qwen2.5-vl technical report. *arXiv:2502.13923*, 2025. 3, 4
- [4] K. Batzner, L. Heckler, and R. König. Efficientad: Accurate visual anomaly detection at millisecond-level latencies. In *WACV*, pages 128–138, 2024. 2
- [5] P. Bergmann, M. Fauser, D. Sattlegger, and C. Steger. Mvtec ad — a comprehensive real-world dataset for unsupervised anomaly detection. In *CVPR*, pages 9592–9600, 2019. 1, 2
- [6] P. J. Burt and E. H. Adelson. The laplacian pyramid as a compact image code. *IEEE Trans. Communications*, 31(4): 532–540, 1983. 5
- [7] J. Canny. A computational approach to edge detection. *IEEE Trans. Pattern Anal. Mach. Intell.*, PAMI-8(6):679–698, 1986. 4
- [8] Q. Chen, H. Luo, C. Lv, and Z. Zhang. A unified anomaly synthesis strategy with gradient ascent for industrial anomaly detection and localization. In *ECCV*, pages 37–54, 2024. 2
- [9] Z. Dai, S. Zeng, H. Liu, X. Li, F. Xue, and Y. Zhou. Seas: Few-shot industrial anomaly image generation with separation and sharing fine-tuning. In *ICCV*, 2025. 2
- [10] Simon Damm, Mike Laszkiewicz, Johannes Lederer, and Asja Fischer. AnomalyDINO: Boosting patch-based few-shot anomaly detection with DINOv2. In *WACV*, 2025. 2, 7
- [11] T. Defard, A. Setkov, A. Loesch, and R. Audigier. Padim: A patch distribution modeling framework for anomaly detection and localization. In *ICPR Workshops*, pages 475–489, 2021. 2, 7
- [12] H. Deng and X. Li. Anomaly detection via reverse distillation from one-class embedding. In *CVPR*, pages 9737–9746, 2022. 2
- [13] Y. Duan, Y. Hong, L. Niu, and L. Zhang. Few-shot defect image generation via defect-aware feature manipulation. In *AAAI*, pages 571–578, 2023. 2
- [14] Georgios D. Evangelidis and Emmanouil Z. Psarakis. Parametric image alignment using enhanced correlation coefficient maximization. *IEEE Trans. Pattern Anal. Mach. Intell.*, 30(10):1858–1865, 2008. 4
- [15] Gunnar Farneback. Two-frame motion estimation based on polynomial expansion. In *SCIA*, pages 363–370, 2003. 4
- [16] A. Fournier, D. Fussell, and L. Carpenter. Computer rendering of stochastic models. *Communications of the ACM*, 25(6):371–384, 1982. 5
- [17] Z. Gu, B. Zhu, G. Zhu, Y. Chen, M. Tang, and J. Wang. Anomalygpt: Detecting industrial anomalies using large vision-language models. In *AAAI*, pages 1932–1940, 2024. 2
- [18] G. Gui, B.-B. Gao, J. Liu, C. Wang, and Y. Wu. Few-shot anomaly-driven generation for anomaly classification and segmentation. In *ECCV*, pages 210–226, 2024. 2
- [19] Jia Guo, Shuai Lu, Weihang Zhang, Fang Chen, Huiqi Li, and Hongen Liao. Dinomaly: The less is more philosophy in multi-class unsupervised anomaly detection. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, pages 20405–20415, 2025. 2, 7
- [20] L. Heckler-Kram, J.-H. Neudeck, U. Scheler, R. König, and C. Steger. The mvtec ad 2 dataset: Advanced scenarios for unsupervised anomaly detection. *arXiv:2503.21622*, 2025. 1, 2, 7
- [21] J. Hu, F. Borsatti, A. Stropeni, D. Dalle Pezze, M. Barusco, and G. A. Susto. Mirage: Model-agnostic industrial realistic anomaly generation and evaluation. *arXiv:2603.13507*, 2026. 2
- [22] T. Hu et al. Anomalydiffusion: Few-shot anomaly image generation with diffusion model. In *AAAI*, pages 8526–8534, 2024. 1, 2
- [23] J. Jeong, Y. Zou, T. Kim, D. Zhang, A. Ravichandran, and O. Dabeer. Winclip: Zero-/few-shot anomaly classification and segmentation. In *CVPR*, pages 19606–19616, 2023. 2
- [24] J. Jia, J. Sun, C.-K. Tang, and H.-Y. Shum. Drag-and-drop pasting. *ACM Trans. Graph.*, 25(3):631–637, 2006. 6
- [25] Y. Jiang et al. Anomagic: Crossmodal prompt-driven zero-shot anomaly generation. In *AAAI*, pages 5485–5493, 2026. 2
- [26] Y. Jin et al. Dual-interrelated diffusion model for few-shot anomaly image generation. In *CVPR*, 2025. 1, 2
- [27] Z. Lai et al. Anomalypainter: Vision-language-diffusion synergy for realistic and diverse unseen industrial anomaly synthesis. In *AAAI*, pages 5800–5808, 2026. 2
- [28] J. P. Lewis. Algorithms for solid noise synthesis. *ACM SIGGRAPH Computer Graphics*, 23(3):263–270, 1989. 5
- [29] C.-L. Li, K. Sohn, J. Yoon, and T. Pfister. Cutpaste: Self-supervised learning for anomaly detection and localization. In *CVPR*, pages 9664–9674, 2021. 1, 2
- [30] Z. Liu, Y. Zhou, Y. Xu, and Z. Wang. Simplenet: A simple network for image anomaly detection and localization. In *CVPR*, pages 20402–20411, 2023. 1, 2
- [31] D. G. Lowe. Distinctive image features from scale-invariant keypoints. *Int. J. Comput. Vis.*, 60(2):91–110, 2004. 4
- [32] S. Niu, B. Li, X. Wang, and H. Lin. Defect image sample generation with gan for improving defect recognition. *IEEE Trans. Automation Science and Engineering*, 17(3): 1611–1622, 2020. 2
- [33] OpenAI. GPT-4o system card. *arXiv:2410.21276*, 2024. 3
- [34] N. Otsu. A threshold selection method from gray-level histograms. *IEEE Trans. Systems, Man, and Cybernetics*, 9(1): 62–66, 1979. 5
- [35] K. Perlin. An image synthesizer. In *SIGGRAPH*, pages 287–296, 1985. 2, 5, 7
- [36] T. Porter and T. Duff. Compositing digital images. *ACM SIGGRAPH Computer Graphics*, 18(3):253–259, 1984. 6
- [37] P. Pérez, M. Gangnet, and A. Blake. Poisson image editing. *ACM Trans. Graph.*, 22(3):313–318, 2003. 6
- [38] Erik Reinhard, Michael Ashikhmin, Bruce Gooch, and Peter Shirley. Color transfer between images. *IEEE Computer Graphics and Applications*, 21(5):34–41, 2001. 6
- [39] R. Rombach, A. Blattmann, D. Lorenz, P. Esser, and B. Ommer. High-resolution image synthesis with latent diffusion models. In *CVPR*, pages 10684–10695, 2022. 2
- [40] Lukas Roming, Felix Lehnerer, Jonas V. Funk, Andreas Michel, Georg Maier, Thomas Längle, and Jürgen Beyerer.

- 643 SuperADD: Training-free class-agnostic anomaly segmenta-
644 tion. In *CVPR Workshops (VAND 4.0 Challenge, Industrial*
645 *Track)*, 2026. [2](#), [7](#), [8](#)
- 646 [41] A. Rosenfeld and J. L. Pfaltz. Sequential operations in digi-
647 tal picture processing. *Journal of the ACM*, 13(4):471–494,
648 1966. [6](#)
- 649 [42] K. Roth, L. Pemula, J. Zepeda, B. Schölkopf, T. Brox, and P.
650 Gehler. Towards total recall in industrial anomaly detection.
651 In *CVPR*, pages 14318–14328, 2022. [2](#), [7](#)
- 652 [43] H. M. Schlüter, J. Tan, B. Hou, and B. Kainz. Natural syn-
653 thetic anomalies for self-supervised anomaly detection and
654 localization. In *ECCV*, pages 474–489, 2022. [2](#)
- 655 [44] J. Serra. *Image Analysis and Mathematical Morphology*.
656 Academic Press, 1982. [5](#)
- 657 [45] Q. Shi, J. Wei, F. Shen, and Z. Zhang. Few-shot defect image
658 generation based on consistency modeling. In *ECCV*, 2024.
659 [2](#)
- 660 [46] Oriane Siméoni, Huy V. Vo, Maximilian Seitzer, Fed-
661 erico Baldassarre, Maxime Oquab, et al. DINOv3.
662 *arXiv:2508.10104*, 2025. [7](#)
- 663 [47] Yulim So and Seokho Kang. AnoStyler: Text-driven local-
664 ized anomaly generation via lightweight style transfer. In
665 *AAAI*, 2026. *arXiv:2511.06687*. [7](#)
- 666 [48] J. Song et al. Defectfill: Realistic defect generation with
667 inpainting diffusion model for visual inspection. In *CVPR*,
668 pages 18718–18727, 2025. [2](#), [3](#)
- 669 [49] J. Tan, B. Hou, J. Batten, H. Qiu, and B. Kainz. Detecting
670 outliers with foreign patch interpolation. *Machine Learning*
671 *for Biomedical Imaging*, 1:1–27, 2022. [2](#)
- 672 [50] X. Xu et al. A survey on industrial anomalies synthesis.
673 *arXiv:2502.16412*, 2025. [1](#)
- 674 [51] M. Yang, P. Wu, and H. Feng. Memseg: A semi-supervised
675 method for image surface defect detection using differences
676 and commonalities. *Engineering Applications of Artificial*
677 *Intelligence*, 119:105835, 2023. [2](#)
- 678 [52] V. Zavrtanik, M. Kristan, and D. Skočaj. Draem — a dis-
679 criminatively trained reconstruction embedding for surface
680 anomaly detection. In *ICCV*, pages 8330–8339, 2021. [1](#), [2](#),
681 [7](#)
- 682 [53] G. Zhang, K. Cui, T.-Y. Hung, and S. Lu. Defect-gan: High-
683 fidelity defect synthesis for automated defect inspection. In
684 *WACV*, pages 2524–2534, 2021. [2](#)
- 685 [54] Q. Zhang, S. Zhang, J. Liu, et al. Asbench: Im-
686 age anomalies synthesis benchmark for anomaly detection.
687 *arXiv:2510.07927*, 2025. [1](#)
- 688 [55] X. Zhang, M. Xu, and X. Zhou. Realnet: A feature selection
689 network with realistic synthetic anomaly for anomaly detec-
690 tion. In *CVPR*, pages 16699–16708, 2024. [2](#)
- 691 [56] Q. Zhou, G. Pang, Y. Tian, S. He, and J. Chen. Anomaly-
692 clip: Object-agnostic prompt learning for zero-shot anomaly
693 detection. In *ICLR*, 2024. [2](#)
- 694 [57] Y. Zou, J. Jeong, L. Pemula, D. Zhang, and O. Dabeer. Spot-
695 the-difference self-supervised pre-training for anomaly de-
696 tection and segmentation. In *ECCV*, pages 392–408, 2022.
697 [2](#)